KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

Verslag: Memory management

Geavanceerde computerarchitectuur

Robbe Hovaert
Lukas Hoste

Master in de ingenieurswetenschappen

Academiejaar 2024-2025

Context

Het doel van dit labo is om kennis te maken met memory management op de GPU. Hiervoor wordt ten eerste naar coalesced memory gekeken. Om dit te bekijken wordt opnieuw een inversie op een foto gedaan, deze keer enkel op het rode kanaal. Ten tweede worden de verschillende types memory onderzocht, wat we doen door een matrix multiplicatie te implementeren aan de hand van global memory, shared memory en constant memory.

Methodologie

Coalesced memory

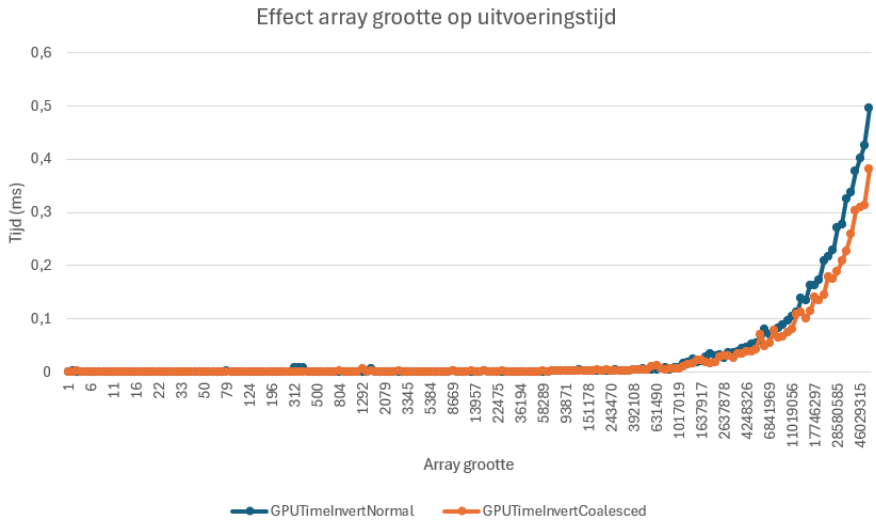
Als eerste experiment wordt een inversie gedaan op een foto, beginnend in de vorm RGBRGB...RGB. Hierop wordt inversie op het rode kanaal gedaan. Om dit te bereiken wordt de threadId vermenigvuldigd met drie, om zo alleen de rode pixels te selecteren. In de coalesced variant wordt de inversie toegepast op een foto van de vorm RRR...GGG...BBB. Hierin voert elke thread een inversie berekening uit tot het einde van het rode kanaal bereikt wordt $(N/3)$.

Matrix vermenigvuldiging

Als tweede experiment wordt een matrix vermenigvuldiging uitgevoerd. Dit doen we eerst aan de hand van global memory. Hierin bepalen we als eerste hoeveel elementen berekend moeten worden (kolommen x rijen). Vervolgens wordt een loop met striding gestart die stopt wanneer deze hoeveelheid elementen bereikt wordt. Vervolgens wordt de rij en kolom berekend in de matrix C die de huidige thread gaat invullen, met rij = tid/C en kolom = $tid\%C$ met C het aantal kolommen. Hierna kan de som van producten berekend worden met deze rij (matrix A) en kolom (matrix B). Ten laatste wordt deze som in de resultatenmatrix(C) geplaatst.

Als tweede wordt de matrix multiplicatie uitgevoerd aan de hand van constant memory. Hierin wordt dezelfde berekening uitgevoerd als in de versie met global memory, maar wordt voor matrix A en B constant memory gebruikt.

Als laatste wordt de matrix multiplicatie uitgevoerd aan de hand van shared memory. In deze implementatie wordt als eerste het shared memory aangemaakt. Dit zijn 2D-arrays voor een submatrix van matrix A en B, deze zijn BLOCK_SIZE x BLOCK_SIZE groot. Vervolgens worden de thread ids (x en y), rij en kolom berekend. Hierin is de rij = $blockIdx.y * BLOCK_SIZE + tidy$ en kolom = $blockIdx.x * BLOCK_SIZE + tidx$. Vervolgens kan de som berekend worden, hiervoor wordt over alle blocks/sub matrices van A en B gegaan. Hier moeten als eerste alle benodigde elementen in het shared memory ingeladen worden. Na het inladen van dit geheugen worden de threads gesynchroniseerd zodat de effectieve berekening pas gebeurt van zodra alle data ingeladen is. Hierna wordt de standaard som van producten berekening uitgevoerd aan de hand van het shared memory en worden de threads opnieuw gesynchroniseerd. Dit wordt herhaald totdat alle



Figuur 1: Array grootte tegenover executietijd in ms (gemiddelde van 50 uitvoeringen)

blocks/sub matrices overlopen zijn. Ten laatste wordt het resultaat naar de resultatenmatrix geschreven (global memory).

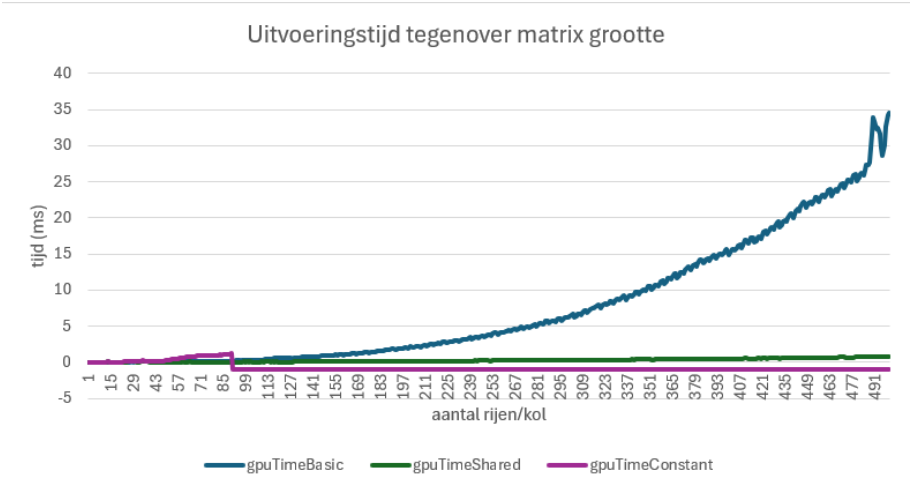
Observaties en resultaten

Wij hebben aan de bovenstaande implementatie timing-informatie toegevoegd. Wij houden de tijd dat de GPU nodig heeft om dit uit te voeren (tijd zonder memcpy) bij (GPU Time) en de tijd op de CPU om de data te verplaatsen (bij constant memory wordt dit niet meegeteld) en de operatie op de gpu uit te voeren (GPU CPU Time). Ten laatste wordt de tijd om van cpu naar global memory (cpuGpuCopyTime) en om van global naar shared memory (gpuGlobalSharedCopyTime) te gaan bijgehouden.

Als eerste werd het effect van coalesced memory getest (Figuur 1). Hiervoor wordt de uitvoeringstijd van beide implementaties van de inverting kernel, met variatie van de array grootte getest. Uit de resultaten zien we dat er bij kleine array groottes bijna geen verschil is tussen de uitvoeringstijd. Vanaf grootte arrays begint het effect van coalesced memory echter duidelijk te worden en is het duidelijk te zien dat de versie die coalesced memory gebruikt een stuk sneller is. Dit is ook het resultaat dat we verwachten te zien.

Als tweede werd het effect van de matrix grootte op de verschillende methoden voor matrix multiplicatie getest (Figuur 2). Zoals verwacht is bij kleinere matrix groottes de versie met constant memory het snelste, van zodra de matrix grootte echter groter wordt begint deze methode ook minder efficient te worden. Daarnaast kan deze bij matrix groottes groter dan 90x90 zelf helemaal niet meer gebruikt worden. Daarnaast zien we ook dat de versie met global memory op een gegeven moment lineair begint te stijgen. De shared memory versie blijft echter vrij constant en is dus voor grote matrices veel efficienter.

Bij de matrix multiplicatie met shared memory hebben wij ook testen gedaan op de invloed van de matrix-grootte op de kopieertijd tussen de geheugens (Figuur 3). Deze stijgt zowel voor de kopieertijd van de CPU naar het globaal GPU memory als voor de kopieertijd van de GPU globaal memory naar het shared memory linear. We merken wel op dat de tijd om naar het shared geheugen te kopiëren veel lager is dan voor naar het globaal



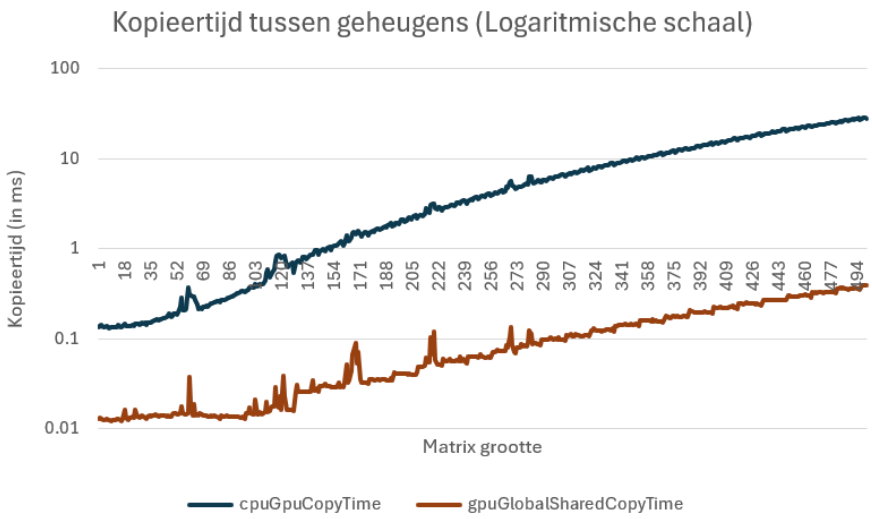
Figuur 2: Matrix grootte tegenover executietijd in ms (gemiddelde van 50 uitvoeringen)

geheugen. Deze tijd heeft dus weinig impact op de uiteindelijke uivoeringstijd van het algoritme.

Conclusie

In dit labo hebben wij het effect onderzocht van coalesced memory en de verschillen in het gebruik van de verschillende soorten memory. We merkten op dat het geheugen op een coalesced manier gebruiken beter is voor de performantie. Hier moet wel nog rekening gehouden worden met pre en post-processing times, aangezien bijvoorbeeld afbeeldingen een coalesced voorstelling niet natuurlijk is.

Als we kijken naar de performantie bij het matrix vermenigvuldigen merkten we op dat global memory op de GPU best traag is en slecht schaalt. Het gebruik maken van constant geheugen is de eenvoudigste optie om veel snelheidswinst te maken, maar deze is beperkt in zijn grootte, waardoor het verschil tegenover globaal geheugen waar constant geheugen bruikbaar is beperkt is. Shared geheugen lijkt zeker voor duure operaties die heel veel moeten lezen en schrijven de beste keuze. Hierbij moeten we wel rekening houden met het feit dat het gebruiken van shared geheugen veel complexer is. Ten laatste leren we dat het overzetten van globaal naar shared memory erg rap is en dus een klein effect heeft op de uiteindelijke uitvoeringstijd. Doordat toegang tot shared memory veel sneller is is deze kopieertijd zelf verwaarloosbaar tegenover de versnelling die optreed door het gebruiken van shared memory.



Figuur 3: Matrix grootte tegenover tijd om geheugens te kopieren in ms (gemiddelde van 50 uitvoeringen

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run `convert <name>.<extension>
 ↪  <name>.ppm
 *
 */
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <chrono>

// #define M 512       // Lenna width
// #define N 512       // Lenna height
#define M 960       // VR width
#define N 1280       // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length

#define ARR_SIZE M * N * C


uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the
     ↪  image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0},
     ↪  ..., b_{b,M}, ..., b_{N,M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./anna.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
```

```
    uint8_t* image_array =
    ↪  (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET,
    ↪  imageFile);

    // Close the file
    fclose(imageFile);

    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}


void save_image_array(uint8_t* image_array){
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./output_image.ppm","wb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile,"P6\n");               // P6 filetype
    fprintf(imageFile,"%d %d\n", M, N);      // dimensions
    fprintf(imageFile,"255\n");              // Max pixel

    // Write the image
    fwrite(image_array, 1, M*N*C, imageFile);

    // Close the file
    fclose(imageFile);
}

__global__ void __invertImgRedGPUNormal(uint8_t* arr, int n) {
    for (int tid = (blockIdx.x * blockDim.x + threadIdx.x) * 3;
        tid < n;
        tid += (blockDim.x * gridDim.x) * 3)
     {
        arr[tid] = 255 - arr[tid];
```

```
    }
}


void invertImgRedGPUNormal(uint8_t* src, uint8_t* dst, int n, int
↪  blockSize, int amountOfBlocks, float * gpuTime, float * gpuCpuTime) {
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    const auto startCPU = std::chrono::steady_clock::now();

        // Allocate memory on device
    uint8_t *gpuSrc;
    cudaMalloc(&gpuSrc, n * sizeof(uint8_t));
    cudaMemcpy(gpuSrc, src, n * sizeof(uint8_t), cudaMemcpyHostToDevice);
    cudaEventRecord(start);
    __invertImgRedGPUNormal<<<amountOfBlocks, blockSize>>>(gpuSrc, n);
    cudaDeviceSynchronize(); // Ensure kernel execution completes
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaMemcpy(dst, gpuSrc, n * sizeof(uint8_t), cudaMemcpyDeviceToHost);
    cudaFree(gpuSrc);

    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    *gpuCpuTime = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
    *gpuTime = milliseconds;

}

void toBlockRGB(uint8_t* src, uint8_t* dst, int n) {
    // on CPU, convert to RRRRRRRRRGGGGGGGGGBBBBBBBBB
    // first all the red values
    int indexSrc = 0;
    int indexDst = 0;
    while (indexSrc < n) {
        dst[indexDst] = src[indexSrc];
        indexSrc += 3;
        indexDst++;
    }
    // then all the green values
    indexSrc = 1;
    while (indexSrc < n) {
        dst[indexDst] = src[indexSrc];
        indexSrc += 3;
        indexDst++;
```

```
    }
    // then all the blue values
    indexSrc = 2;
    while (indexSrc < n) {
        dst[indexDst] = src[indexSrc];
        indexSrc += 3;
        indexDst++;
    }
}

void fromBlockRgb(uint8_t* src, uint8_t* dst, int n) {
    // Convert back to RGBRGB
    int indexSrc = 0;
    int indexDst = 0;

    // Process the red values
    while (indexSrc < n / 3) {
        dst[indexDst] = src[indexSrc];
        indexSrc++;
        indexDst += 3;  // Move to the next RGB position
    }

    // Process the green values
    indexSrc = n / 3;  // Start from the green block
    indexDst = 1;
    while (indexSrc < 2 * n / 3) {
        dst[indexDst] = src[indexSrc];
        indexSrc++;
        indexDst += 3;  // Move to the next RGB position
    }

    // Process the blue values
    indexSrc = 2 * n / 3;  // Start from the blue block
    indexDst = 2;
    while (indexSrc < n) {
        dst[indexDst] = src[indexSrc];
        indexSrc++;
        indexDst += 3;  // Move to the next RGB position
    }
}

__global__ void __invertImgRedGPUCoalesced(uint8_t* arr, int n) {
    int endRed = n /3 ;
    for (int tid = blockIdx.x * blockDim.x + threadIdx.x;
        tid < endRed;
        tid += blockDim.x * gridDim.x)
     {
        if (tid < endRed) {
            arr[tid] = 255 - arr[tid];
```

```
        }
    }
}


void invertImgRedGPUCoalesced(uint8_t* src, uint8_t* dst, int n, int
↪  blockSize, int amountOfBlocks, float * gpuTime, float * gpuCpuTime) {
    uint8_t* blockSrc = (uint8_t*)malloc(sizeof(uint8_t) * n);
    uint8_t* blockDst = (uint8_t*)malloc(sizeof(uint8_t) * n);
    toBlockRGB(src, blockSrc, n);

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    const auto startCPU = std::chrono::steady_clock::now();

        // Allocate memory on device
    uint8_t *gpuSrc;
    cudaMalloc(&gpuSrc, n * sizeof(uint8_t));
    cudaMemcpy(gpuSrc, blockSrc, n * sizeof(uint8_t),
    ↪  cudaMemcpyHostToDevice);
    cudaEventRecord(start);
    __invertImgRedGPUCoalesced<<<amountOfBlocks, blockSize>>>(gpuSrc, n);
    cudaDeviceSynchronize(); // Ensure kernel execution completes
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaMemcpy(blockDst, gpuSrc, n * sizeof(uint8_t),
    ↪  cudaMemcpyDeviceToHost);
    cudaFree(gpuSrc);

    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    *gpuCpuTime = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
    *gpuTime = milliseconds;

    fromBlockRgb(blockDst, dst, n);
    free(blockSrc);
    free(blockDst);

}
```

```
size_t getMaxAllocatableMemory() {
    size_t freeMem, totalMem;
    cudaMemGetInfo(&freeMem, &totalMem);
    return freeMem / 2;  // Use half to avoid fragmentation issues
}

int main (void) {

    size_t maxSize = getMaxAllocatableMemory();
    size_t numElements = maxSize / sizeof(uint8_t);  // Number of
    ↪  elements we can allocate

    // printf("Max array size: %zu bytes (%zu elements)\n", maxSize,
    ↪  numElements);

    uint8_t* src = (uint8_t*)malloc(numElements);
    uint8_t* dst = (uint8_t*)malloc(numElements);

    // Fill with random values
    for (size_t i = 0; i < numElements; i++) {
        src[i] = rand() % 256;
    }

    // Read the image
    uint8_t* image_array = get_image_array();

    // Allocate output
    uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);
    float gpuTime = 0;
    float cpuTime = 0;
    //invertImgRedGPUNormal(image_array, new_image_array, ARR_SIZE, 1,
    ↪  16, &gpuTime, &cpuTime);
    invertImgRedGPUCoalesced(image_array, new_image_array, ARR_SIZE, 1,
    ↪  16, &gpuTime, &cpuTime);
    save_image_array(new_image_array);
    // printf("Done!\n");
    /*// Convert to grayscale using only the red color component
    for(int i=0; i<M*N*C; i++){
        new_image_array[i] = image_array[i/3*3];
    }*/

    // execute once to warm up
    // invertImgGPU(image_array, new_image_array, ARR_SIZE, 1, 16,
    ↪  &gpuInvertTime, &gpuCpuInvertTime);
    // invoke the inverse kernel from loop, gradually increase the amount
    ↪  of threads
    // we do this to see the effect of the warps
    // per amount of threads do 100 iterations and take an average
```

```
//
↪  printf("ThreadsPerBlock,GPUTimeInvertNormal,CPUGPUTimeInvertNormal,GPUTimeInvertC
// for (int i = 1; i <= 1024; i+=2) {

//     float gpuInvertTimeNormal = 0;
//     float gpuCpuInvertTimeNormal = 0;

//     float gpuInvertTimeCoalesced = 0;
//     float gpuCpuInvertTimeCoalesced = 0;

//     for (int j = 0; j < 50; j++) {
//         invertImgRedGPUNormal(image_array, new_image_array,
↪  ARR_SIZE, i, 16, &gpuInvertTimeNormal, &gpuCpuInvertTimeNormal);
//         invertImgRedGPUCoalesced(image_array, new_image_array,
↪  ARR_SIZE, i, 16, &gpuInvertTimeCoalesced,
↪  &gpuCpuInvertTimeCoalesced);
//     }
//     printf("%d,%f,%f,%f,%f\n", i, gpuInvertTimeNormal / 50,
↪  gpuCpuInvertTimeNormal / 50, gpuInvertTimeCoalesced / 50,
↪  gpuCpuInvertTimeCoalesced / 50);
//     //printf("%d, %f, %f, %d, %d, %d\n", i, gpuTime / 50,
↪  gpuCpuTime / 50, warps, idleCores, usedCores);
// }

// timing of different input sizes

↪  printf("ArraySize,GPUTimeInvertNormal,CPUGPUTimeInvertNormal,GPUTimeInvertCoalesc

for(int i = 1; i < 46029315; i = max(static_cast<int>(i * 1.1), i +
↪  1)) {
    float gpuInvertTimeNormal = 0;
    float gpuCpuInvertTimeNormal = 0;

    float gpuInvertTimeCoalesced = 0;
    float gpuCpuInvertTimeCoalesced = 0;

    for(int j = 0; j < 50; j++) {
        // invert the img up till index i
        invertImgRedGPUNormal(src, dst, i, 1024, 32,
        ↪  &gpuInvertTimeNormal, &gpuCpuInvertTimeNormal);
        invertImgRedGPUCoalesced(src, dst, i, 1024, 32,
        ↪  &gpuInvertTimeCoalesced, &gpuCpuInvertTimeCoalesced);
    }
    printf("%d,%f,%f,%f,%f\n", i, gpuInvertTimeNormal / 50,
    ↪  gpuCpuInvertTimeNormal / 50, gpuInvertTimeCoalesced / 50,
    ↪  gpuCpuInvertTimeCoalesced / 50);

}
```

```
    free(src);
    free(dst);



//    blurImgGPU(image_array, new_image_array, ARR_SIZE, 2, 16);
   //invertImgGPU(image_array, new_image_array, ARR_SIZE ,1, 16);

    // Save the image
   //blurImgGPU(image_array, new_image_array, ARR_SIZE, 2, 16);
   //blockBlurImgGPU(image_array, new_image_array, ARR_SIZE, 2, 16);
    //save_image_array(new_image_array);
    //printf("Done!\n");
    //printf("%d", sizeof(uint8_t));
    free(new_image_array);
    return 0;
}
```

B CODE MATRIX

```
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <chrono>

#define MAX_MATRIX_SIZE 90
#define BLOCK_SIZE 16

// declare constant memory matrices
__constant__ int const_a[MAX_MATRIX_SIZE * MAX_MATRIX_SIZE];
__constant__ int const_b[MAX_MATRIX_SIZE * MAX_MATRIX_SIZE];

// Generate a random number
unsigned int rand_custom(unsigned int* seed) {
    // Constants for LCG (same as ANSI C `rand`)
    (*seed) = (1103515245 * (*seed) + 12345) & 0x7fffffff;  // Keep it
    ↪  31-bit
    return (*seed);
}

// Generate a random number within a range [min, max]
int rand_range(int min, int max, unsigned int* seed) {
    if (min > max) {  // Swap if min is greater than max
        int temp = min;
        min = max;
        max = temp;
    }
    unsigned int range = max - min + 1;
    return (rand_custom(seed) % range) + min;
}

struct matrix {
    int cols;
    int rows;
    int* elements;
};


matrix create_matrix(int rows, int cols) {
    matrix m;
    m.cols = cols;
    m.rows = rows;
    m.elements = (int*)malloc(cols * rows * sizeof(int));
    return m;
}
```

```
matrix create_random_matrix(int rows, int cols, unsigned int* seed) {
    matrix m = create_matrix(rows, cols);
    for (int i = 0; i < cols * rows; i++) {
        m.elements[i] = rand_range(0, 25, seed);
    }
    return m;
}

void free_matrix(matrix m) {
    free(m.elements);
}

void print_matrix(matrix m) {
    for (int i = 0; i < m.rows; i++) {
        for (int j = 0; j < m.cols; j++) {
            printf("%3d ", m.elements[i * m.cols + j]);
        }
        printf("\n");
    }
}

bool are_matrixes_equal(matrix a, matrix b) {
    if (a.cols != b.cols || a.rows != b.rows) {
        return false;
    }

    for (int i = 0; i < a.cols * a.rows; i++) {
        if (a.elements[i] != b.elements[i]) {
            return false;
        }
    }

    return true;
}



__host__ __device__ int get_matrix_val(matrix m, int row, int col) {
    return m.elements[row * m.cols + col];
}

__host__ __device__ void set_matrix_val(matrix m, int row, int col, int
↪   val) {
    m.elements[row * m.cols + col] = val;
}


void cudaCopyMatrix(matrix src, matrix dst, cudaMemcpyKind direction) {
```

```
    // the caller should ensure that everything is allocated and
    ↪ matching
    cudaMemcpy(dst.elements, src.elements, src.cols * src.rows *
    ↪ sizeof(int), direction);
}

matrix cudaAllocMatrix(int rows, int cols) {
    matrix m;
    m.cols = cols;
    m.rows = rows;
    cudaMalloc(&m.elements, cols * rows * sizeof(int));
    return m;
}

void cudaFreeMatrix(matrix m) {
    cudaFree(m.elements);
}


__global__ void __multiply_matrices_GpuBasic(matrix a, matrix b, matrix
↪ c) {
    int maxElem = c.cols * c.rows;

    for (int tid = blockIdx.x * blockDim.x + threadIdx.x;
        tid < maxElem;
        tid += blockDim.x * gridDim.x)
     {

        int row = tid / c.cols;
        int col = tid % c.cols;
        int sum = 0;
        for (int i = 0; i < a.cols; i++) {
            sum += get_matrix_val(a, row, i) * get_matrix_val(b, i, col);
        }
        set_matrix_val(c, row, col, sum);
     }
}




matrix multiply_matrices_GpuBasic(matrix a, matrix b, int blockSize, int
↪ amountOfBlocks, float * gpuTime, float * cpuTime) {
    // validate the dimensions
    if (a.cols != b.rows) {
        printf("Invalid matrix dimensions\n");
        exit(1);
    }
```

```
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    const auto startCPU = std::chrono::steady_clock::now();

    // Allocate memory for the result matrix
    matrix c = create_matrix(a.rows, b.cols);

    matrix a_GPU = cudaAllocMatrix(a.rows, a.cols);
    matrix b_GPU = cudaAllocMatrix(b.rows, b.cols);
    matrix c_GPU = cudaAllocMatrix(c.rows, c.cols);

    // Copy the matrices to the GPU
    cudaCopyMatrix(a, a_GPU, cudaMemcpyHostToDevice);
    cudaCopyMatrix(b, b_GPU, cudaMemcpyHostToDevice);

    cudaEventRecord(start);

    __multiply_matrices_GpuBasic<<<amountOfBlocks, blockSize>>>(a_GPU,
    ↪  b_GPU, c_GPU);
    cudaDeviceSynchronize();

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    // Copy the result matrix back to the CPU
    cudaCopyMatrix(c_GPU, c, cudaMemcpyDeviceToHost);

    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    *cpuTime = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
    *gpuTime = milliseconds;

    // Free the memory
    cudaFreeMatrix(a_GPU);
    cudaFreeMatrix(b_GPU);
    cudaFreeMatrix(c_GPU);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return c;
}
__global__ void __multiply_matrices_GpuConstant(matrix c) {
    int maxElem = c.cols * c.rows;
```

```
    for (int tid = blockIdx.x * blockDim.x + threadIdx.x;
        tid < maxElem;
        tid += blockDim.x * gridDim.x)
    {
        int row = tid / c.cols;
        int col = tid % c.cols;
        int sum = 0;
        for (int i = 0; i < c.cols; i++) {
            // Use constant memory for matrix a and b
            sum += const_a[row * c.cols + i] * const_b[i * c.cols + col];
        }
        set_matrix_val(c, row, col, sum);
    }
}

matrix multiply_matrices_GpuConstant(matrix a, matrix b, int blockSize,
↪   int amountOfBlocks, float * gpuTime, float * cpuTime) {
    // Validate the dimensions
    if (a.cols != b.rows) {
        printf("Invalid matrix dimensions\n");
        exit(1);
    }

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    // Allocate memory for the result matrix
    matrix c = create_matrix(a.rows, b.cols);

    // Copy the matrices to constant memory on the device
    cudaMemcpyToSymbol(const_a, a.elements, a.rows * a.cols *
    ↪   sizeof(int));
    cudaMemcpyToSymbol(const_b, b.elements, b.rows * b.cols *
    ↪   sizeof(int));

    // TODO: mss logischer hier te zetten aangezien in realistische
    ↪   situatie de matrices niet veranderen
    const auto startCPU = std::chrono::steady_clock::now();

    matrix c_GPU = cudaAllocMatrix(c.rows, c.cols);

    cudaEventRecord(start);

    // Launch the kernel
    __multiply_matrices_GpuConstant<<<amountOfBlocks,
    ↪   blockSize>>>(c_GPU);
    cudaDeviceSynchronize();
```

```
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    // Copy the result matrix back to the CPU
    cudaCopyMatrix(c_GPU, c, cudaMemcpyDeviceToHost);

    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    *cpuTime = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
    *gpuTime = milliseconds;

    // Free the memory
    cudaFreeMatrix(c_GPU);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return c;
}

__global__ void matrixMulShared(matrix a, matrix b, matrix c, unsigned
↪  long long *copyGpuCycles) {
    // Allocate shared memory for sub-matrices of A and B
    __shared__ int shared_a[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ int shared_b[BLOCK_SIZE][BLOCK_SIZE];

    // Calculate thread indices
    int tidx = threadIdx.x;
    int tidy = threadIdx.y;
    int row = blockIdx.y * BLOCK_SIZE + tidy;
    int col = blockIdx.x * BLOCK_SIZE + tidx;

    int sum = 0;
    unsigned long long start = 0;
    unsigned long long end = 0;


    // Loop over all tiles
    for (int i = 0; i < (a.cols + BLOCK_SIZE - 1) / BLOCK_SIZE; ++i) {

        start = clock64();
        // Load elements into shared memory
        if (row < a.rows && (i * BLOCK_SIZE + tidx) < a.cols)
            shared_a[tidy][tidx] = a.elements[row * a.cols + (i *
            ↪  BLOCK_SIZE + tidx)];
        else
```

```
            shared_a[tidy][tidx] = 0;

        if ((i * BLOCK_SIZE + tidy) < b.rows && col < b.cols)
            shared_b[tidy][tidx] = b.elements[(i * BLOCK_SIZE + tidy) *
            ↪  b.cols + col];
        else
            shared_b[tidy][tidx] = 0;
        end = clock64();

        // Synchronize to ensure all threads have loaded their data
        __syncthreads();

        // Perform the multiplication for the current tile
        for (int k = 0; k < BLOCK_SIZE; ++k) {
            sum += shared_a[tidy][k] * shared_b[k][tidx];
        }

        // Synchronize to ensure all threads have completed the
        ↪  computation
        __syncthreads();
    }

    // Write the result to the output matrix
    if (row < c.rows && col < c.cols) {
        c.elements[row * c.cols + col] = sum;
    }

    // get exactly one thread
    if (threadIdx.x == 0 && threadIdx.y == 0 && blockIdx.x == 0 &&
    ↪  blockIdx.y == 0) {
        *copyGpuCycles = end - start;
    }

}

matrix multiply_matrices_GpuShared(matrix a, matrix b, float * gpuTime,
↪  float * cpuTime, float* cpuGPUCopyTime, float*
↪  gpuGlobalGpuSharedCopyTime) {
    // Validate the dimensions
    if (a.cols != b.rows) {
        printf("Invalid matrix dimensions\n");
        exit(1);
    }

    unsigned long long copyGpuCycles_CPUSIDE = 0;
    unsigned long long *copyGpuCycles_GPUSIDE;
    cudaMalloc(&copyGpuCycles_GPUSIDE, sizeof(unsigned long long));
    cudaMemcpy(copyGpuCycles_GPUSIDE, &copyGpuCycles_CPUSIDE,
    ↪  sizeof(unsigned long long), cudaMemcpyHostToDevice);
```

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
const auto startCPU = std::chrono::steady_clock::now();

// Allocate memory for the result matrix
matrix c = create_matrix(a.rows, b.cols);

// Allocate memory on the GPU for matrices A, B, and C
matrix a_GPU = cudaAllocMatrix(a.rows, a.cols);
matrix b_GPU = cudaAllocMatrix(b.rows, b.cols);
matrix c_GPU = cudaAllocMatrix(c.rows, c.cols);

// Copy matrices A and B to the GPU

const auto startMemCpyCPUGPU = std::chrono::steady_clock::now();
cudaCopyMatrix(a, a_GPU, cudaMemcpyHostToDevice);
cudaCopyMatrix(b, b_GPU, cudaMemcpyHostToDevice);
const auto endMemCpyCPUGPU = std::chrono::steady_clock::now();

// Define the block and grid dimensions
dim3 dimBlock(BLOCK_SIZE, BLOCK_SIZE);
dim3 dimGrid((b.cols + BLOCK_SIZE - 1) / BLOCK_SIZE, (a.rows +
↪  BLOCK_SIZE - 1) / BLOCK_SIZE);

cudaEventRecord(start);

// Launch the kernel
matrixMulShared<<<dimGrid, dimBlock>>>(a_GPU, b_GPU, c_GPU,
↪  copyGpuCycles_GPUSIDE);
cudaDeviceSynchronize();

cudaEventRecord(stop);
cudaEventSynchronize(stop);

// Copy the result matrix back to the CPU
cudaCopyMatrix(c_GPU, c, cudaMemcpyDeviceToHost);

const auto endCPU = std::chrono::steady_clock::now();
float milliseconds = 0;
cudaEventElapsedTime(&milliseconds, start, stop);

*cpuTime = std::chrono::duration<double, std::milli>(endCPU -
↪  startCPU).count();
*gpuTime = milliseconds;

// Free the memory on the GPU
cudaFreeMatrix(a_GPU);
cudaFreeMatrix(b_GPU);
```

```
    cudaFreeMatrix(c_GPU);

    // calculate the time it took to copy the data from the CPU to the
    ↪  GPU
    *cpuGPUCopyTime = std::chrono::duration<double,
    ↪  std::milli>(endMemCpyCPUGPU - startMemCpyCPUGPU).count();

    // copy back timing info
    cudaMemcpy(&copyGpuCycles_CPUSIDE, copyGpuCycles_GPUSIDE,
    ↪  sizeof(unsigned long long), cudaMemcpyDeviceToHost);
    cudaFree(copyGpuCycles_GPUSIDE);

    cudaDeviceProp prop;
    cudaGetDeviceProperties(&prop, 0);
    double time_ms = static_cast<double>(copyGpuCycles_CPUSIDE) /
    ↪  prop.clockRate;
    *gpuGlobalGpuSharedCopyTime = time_ms;
    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return c;
}


int main (void) {
    unsigned int seed = 0;
    matrix a = create_random_matrix(16, 16, &seed);
    matrix b = create_random_matrix(16, 16, &seed);

    //print_matrix(a);
    //printf("\n");
    //print_matrix(b);
    //printf("\n");

    float gpuTime = 0;
    float cpuTime = 0;
    float copyTimeCpuGpu = 0;
    float copyTimeGpuGlobalGpuShared = 0;

    matrix d = multiply_matrices_GpuBasic(a, b, 32, 2, &gpuTime,
    ↪  &cpuTime);
    matrix c = multiply_matrices_GpuShared(a, b, &gpuTime, &cpuTime,
    ↪  &copyTimeCpuGpu, &copyTimeGpuGlobalGpuShared);
    //printf("got here\n");
    //print_matrix(c);
    //printf("\n");
    //print_matrix(d);

    // check if elements of c and d are the same
```

```
    // for (int i = 0; i < c.rows; i++) {
    //     for (int j = 0; j < c.cols; j++) {
    //         if (get_matrix_val(c, i, j) != get_matrix_val(d, i, j)) {
    //             printf("Error: c and d are not the same\n");
    //             exit(1);
    //         }
    //     }
    // }

    // timing info to be piped to a csv file
    // we vary the matrix size from 1x1 to 90x90

    ↪  printf("size,gpuTimeBasic,cpuTimeBasic,gpuTimeShared,cpuTimeShared,gpuTimeConstan
    ↪  cpuGpuCopyTime, gpuGlobalSharedCopyTime\n");
    for (int i = 1; i <= 500; i++) {
        matrix a = create_random_matrix(i, i, &seed);
        matrix b = create_random_matrix(i, i, &seed);

        float totalGpuTimeBasic = 0;
        float totalCpuTimeBasic = 0;

        float totalGpuTimeShared = 0;
        float totalCpuTimeShared = 0;

        float totalGpuTimeConstant = 0;
        float totalCpuTimeConstant = 0;

        float totalCpuGPUMemCpyTimeConstant = 0;
        float totalGpuGlobalGpuSharedCopyTime = 0;

        for (int j = 0; j < 50; j++) {
            float gpuTimeBasic = 0;
            float cpuTimeBasic = 0;

            float gpuTimeShared = 0;
            float cpuTimeShared = 0;

            float gpuTimeConstant = 0;
            float cpuTimeConstant = 0;

            float cpuGPUMemCpyTimeConstant = 0;
            float gpuGlobalGpuSharedCopyTime = 0;

            matrix c = multiply_matrices_GpuBasic(a, b, 32, 2,
            ↪  &gpuTimeBasic, &cpuTimeBasic);

            matrix d = multiply_matrices_GpuShared(a, b, &gpuTimeShared,
            ↪  &cpuTimeShared , &cpuGPUMemCpyTimeConstant,
            ↪  &gpuGlobalGpuSharedCopyTime);
```

```
        if (i <= MAX_MATRIX_SIZE) {
            matrix e = multiply_matrices_GpuConstant(a, b, 32, 2,
            ↪  &gpuTimeConstant, &cpuTimeConstant);
            if (!are_matrixes_equal(c, d) || !are_matrixes_equal(c,
            ↪  e)) {
                printf("Error: c, d, and e are not the same\n");
                exit(1);
            }
            free_matrix(e);
        } else {
            gpuTimeConstant = -1;
            cpuTimeConstant = -1;
        }

        totalGpuTimeBasic += gpuTimeBasic;
        totalCpuTimeBasic += cpuTimeBasic;

        totalGpuTimeShared += gpuTimeShared;
        totalCpuTimeShared += cpuTimeShared;

        totalGpuTimeConstant += gpuTimeConstant;
        totalCpuTimeConstant += cpuTimeConstant;

        totalCpuGPUMemCpyTimeConstant += cpuGPUMemCpyTimeConstant;
        totalGpuGlobalGpuSharedCopyTime +=
        ↪  gpuGlobalGpuSharedCopyTime;

        free_matrix(c);
        free_matrix(d);
    }

    printf("%d,%f,%f,%f,%f,%f,%f,%f,%f\n", i,
        totalGpuTimeBasic / 50, totalCpuTimeBasic / 50,
        totalGpuTimeShared / 50, totalCpuTimeShared / 50,
        (i <= MAX_MATRIX_SIZE) ? (totalGpuTimeConstant / 50) : -1,
        (i <= MAX_MATRIX_SIZE) ? (totalCpuTimeConstant / 50) : -1),
        totalCpuGPUMemCpyTimeConstant / 50,
        ↪  totalGpuGlobalGpuSharedCopyTime / 50;

    free_matrix(a);
    free_matrix(b);
}

// matrix c = multiply_matrices_GpuBasic(a, b, 32, 2);
// print_matrix(d);
free_matrix(a);
free_matrix(b);
free_matrix(c);
```

```
    free_matrix(d);
    return 0;
}
```